UNIVERSITÉ LIBRE DE BRUXELLES

# INFO-H420

# Management of Data Science and Business

## Assignment 3: Workflows with Airflow (10 points)

PREPARED FOR

SACHARIDIS Dimitris

PREPARED BY

Group A3.4

SOURATI HASSAN ZADEH Negar

HASHMI Aysha

MAZI Helin

# 1. Introduction

Managing a growing collection of research papers can be time-consuming, particularly when documents must be read, summarized, categorized, and organized manually. Automating these tasks can make research papers easier to manage and retrieve.

The objective of this project is to develop a local, automated pipeline that converts research papers into a searchable digital library using Apache Airflow and a locally hosted Large Language Model (LLM). The system monitors a Google Drive-synchronized folder for new PDF files, converts their content into Markdown, and uses an LLM to extract relevant metadata, including the paper's title, summary, keywords, and category.

The extracted information is stored in a local SQLite database and used to generate a searchable HTML page. Additionally, a Discord notification is sent whenever new papers are successfully processed.

The main focus of the project is workflow orchestration with Apache Airflow, integration with a locally hosted LLM, and automation of the complete processing pipeline.

# 2. System Architecture and Design

## 2.1 Overall Architecture

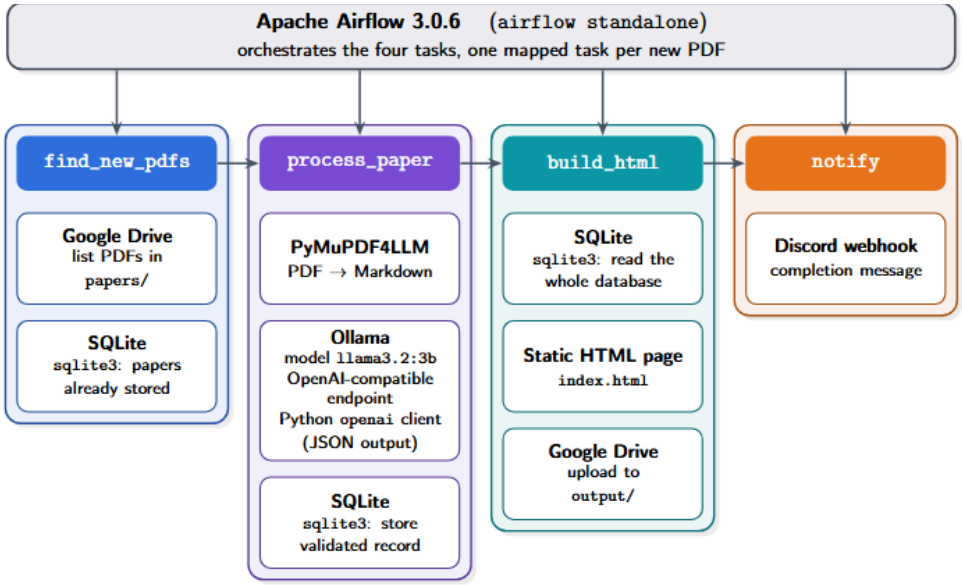
The system consists of several components connected through an Apache Airflow workflow. Google Drive serves as the source of research PDFs and the destination for the generated HTML library, while the SQLite database is maintained locally.

Airflow coordinates the different processing steps, from detecting new documents to generating the HTML page and sending notifications. Each processing operation is implemented as a separate Python function, allowing the components to be developed and tested independently.

We built a small local pipeline that turns research PDFs into a searchable study library. The pipeline works as follows

1. PDF Detection: Detects new PDFs in the Google Drive-synced folder using a scheduled Apache Airflow DAG.
2. PDF Conversion: Convert each PDF into Markdown.
3. LLM Processing: Use a locally hosted LLM to extract the title, summary, keywords and category in JSON format.

4. Database Storage: Validate and store the extracted metadata in a local SQLite database.
5. HTML Publishing: Generate a searchable, category-filterable static HTML page from the database.
6. Discord Notification: Send a Discord notification for each new successfully processed paper.




This figure illustrates how Apache Airflow orchestrates these components and manages the flow of information from PDF detection to the final HTML publication and Discord notification.

## 2.2 Technology Stack

The main technologies used in the implementation are:

- Apache Airflow: Scheduling and workflow orchestration.
- Google Drive: Synchronization of input PDFs and generated HTML.
- PyMuPDF4LLM: Conversion of PDF documents into Markdown.
- Ollama with Llama 3.2 3B: Local LLM inference and metadata extraction.
- SQLite: Local storage of extracted metadata.
- HTML, CSS, and JavaScript: Searchable research-paper library.
- Discord Webhook: Notifications after successful processing.
- Docker: Containerized execution of Apache Airflow.

## 2.3 Data Flow and Storage

The project separates input files, generated outputs, and database storage. Research PDFs are placed in a Google Drive-synchronized folder, while the generated HTML library is saved in a separate output folder. The SQLite database is stored locally, outside Google Drive, as required by the assignment.

This table summarizes the main project components and their locations.

| component | Location |
|---|---|
| Code (DAG and src/) | Cloned GitHub repository (https://github.com/ayshaaaaaaa/INFO-H420_Assignment3) |
| Input PDFs | Google Drive ResearchLibrary/papers/ (PAPERS_DIR) |
| Generated page | Google Drive ResearchLibrary/output/index.html (OUTPUT_DIR) |
| SQLite database | Local machine, ~/airflow_data/papers.db (DB_PATH), outside the synced folder |
| Airflow UI | http://localhost:8080 (The Ui runs locally for everyone) |
| LLM server (Ollama) | http://localhost:11434 |

Google Drive only syncs the input PDFs and the generated HTML. The database stays on the local machine, as the assignment requires.

# 3. Workflow Implementation

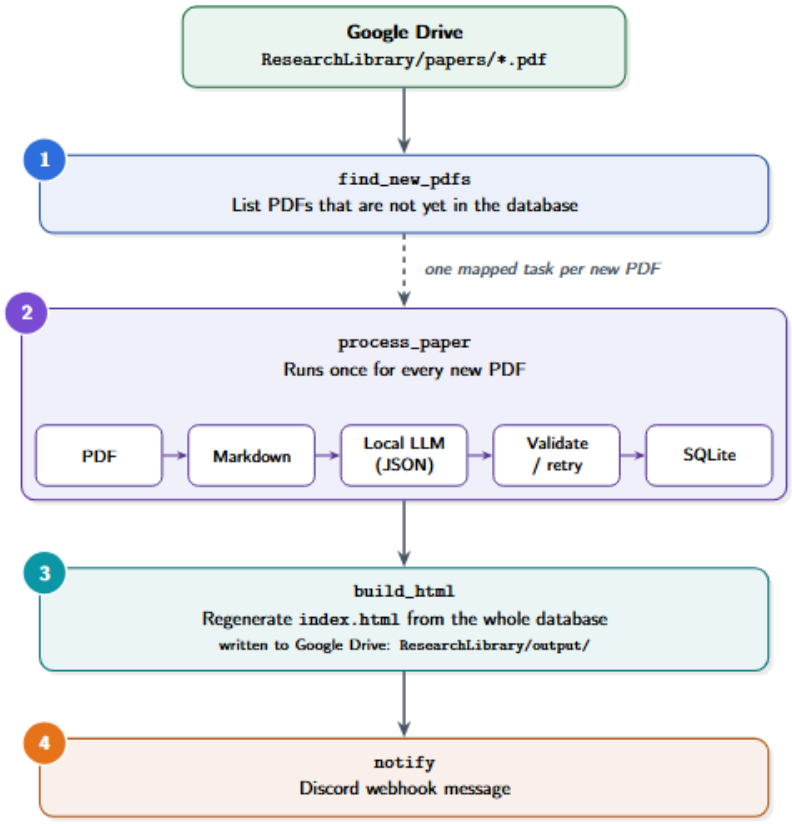
## 3.1 Airflow DAG and Task Orchestration

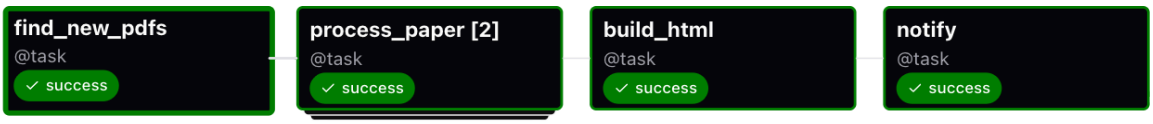
The pipeline is implemented as a single Airflow DAG named paper_pipeline, scheduled to execute every 10 minutes.

It consists of four main tasks:

1. find_new_pdfs: Scans the input directory and identifies PDF files that have not yet been recorded in the SQLite database.
2. process_paper: Converts each new PDF into Markdown, extracts metadata using the local LLM, and saves the results in SQLite.

3. build_html: Retrieves all stored papers and regenerates the HTML library.
4. notify: Sends a Discord notification containing the titles and categories of newly processed papers.




This figure shows the successful execution of the paper_pipeline DAG in Apache Airflow. All four tasks completed successfully, as indicated by their green status indicators. The process_paper [2] task shows that Airflow created two mapped task instances to process the two newly detected PDFs. After processing, the build_html and notify tasks also completed successfully, indicating that the workflow continued through HTML generation and Discord notification without errors.

To ensure reliable and efficient workflow execution, several Airflow configurations were implemented. Following Table  summarizes the main design decisions, including automatic scheduling, dynamic task mapping, sequential processing, retries, and duplicate detection. These configurations allow the pipeline to process new documents automatically while controlling resource usage and avoiding unnecessary task execution.

| Design Choice | What it does | Reason |
|---|---|---|
| Schedule | The DAG runs automatically every [*10 minutes, */10 * * * *]* | No backlog of missed runs is created |
| Dynamic Task Mapping | Creates one task instance per new PDF | Each paper appears as its own instance in the UI and fails or retries independently |
| Sequential Processing | max_active_runs=1 and max_active_tasks=1 | One paper in the LLM at a time |
| Retries and timeout | Retries once and has an execution timeout of 15 mins | A blocking paper does not affect the pipeline |
| Idempotence | find_new_pdfs compares each filename with the database | Avoids duplication |
| Nothing New | If no new PDFs are found then the build_html and notify are skipped | No unnecessary rebuilds |

## 3.2 PDF Conversion and Local LLM Integration

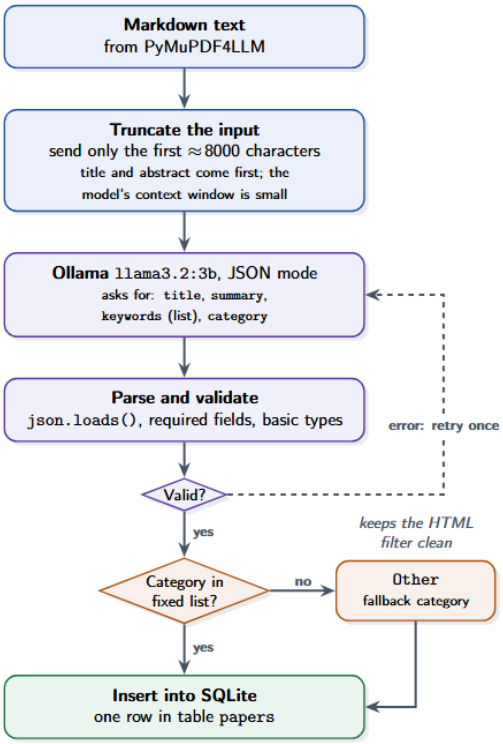
Each newly detected PDF is converted into Markdown using PyMuPDF4LLM. This format preserves useful text structure and prepares the document for metadata extraction.

The converted text is then passed to Llama 3.2 3B, which is served locally through Ollama and accessed using the Python OpenAI client. To limit the input size, only the first 8,000 characters of the converted document are included in the prompt.

The model is instructed to generate a JSON object containing four fields:

- Title: The title of the research paper.
- Summary: A short summary of the paper's content.
- Keywords: A list of relevant research terms.
- Category: The main research area of the paper.

The model is configured to return JSON-formatted responses, simplifying their integration with the subsequent processing steps.



## 3.3 Metadata Validation and SQLite Storage

The LLM response is parsed using Python's json.loads() function. The system verifies that the required fields are present and have the expected basic data types. If parsing or validation fails, the LLM request is repeated once.

To maintain consistent categorization, responses containing categories outside the predefined list are assigned to the Other category.

Validated metadata is stored in a local SQLite database, where each record contains the filename, title, summary, keywords, and category.

Each paper is stored in a row in the SQLite table papers.

| Column | Content | Note |
|---|---|---|
| `filename` | name of the PDF | `UNIQUE`, one row per paper |
| `title` | paper title | from the model output |
| `summary` | short summary | from the model output |
| `keywords` | list of keywords | stored as a JSON string |
| `category` | topic category | from the fixed list, otherwise `Other` |

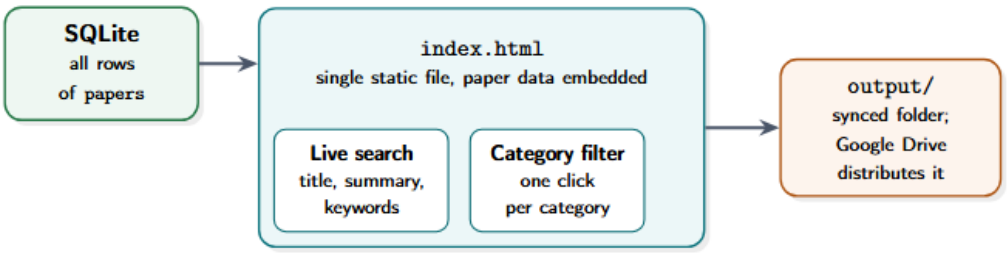
The filename is defined as unique and is also used to identify previously processed papers. This prevents the same document from being processed again during subsequent DAG executions, provided its filename remains unchanged.

SELECT filename, title, category FROM papers;

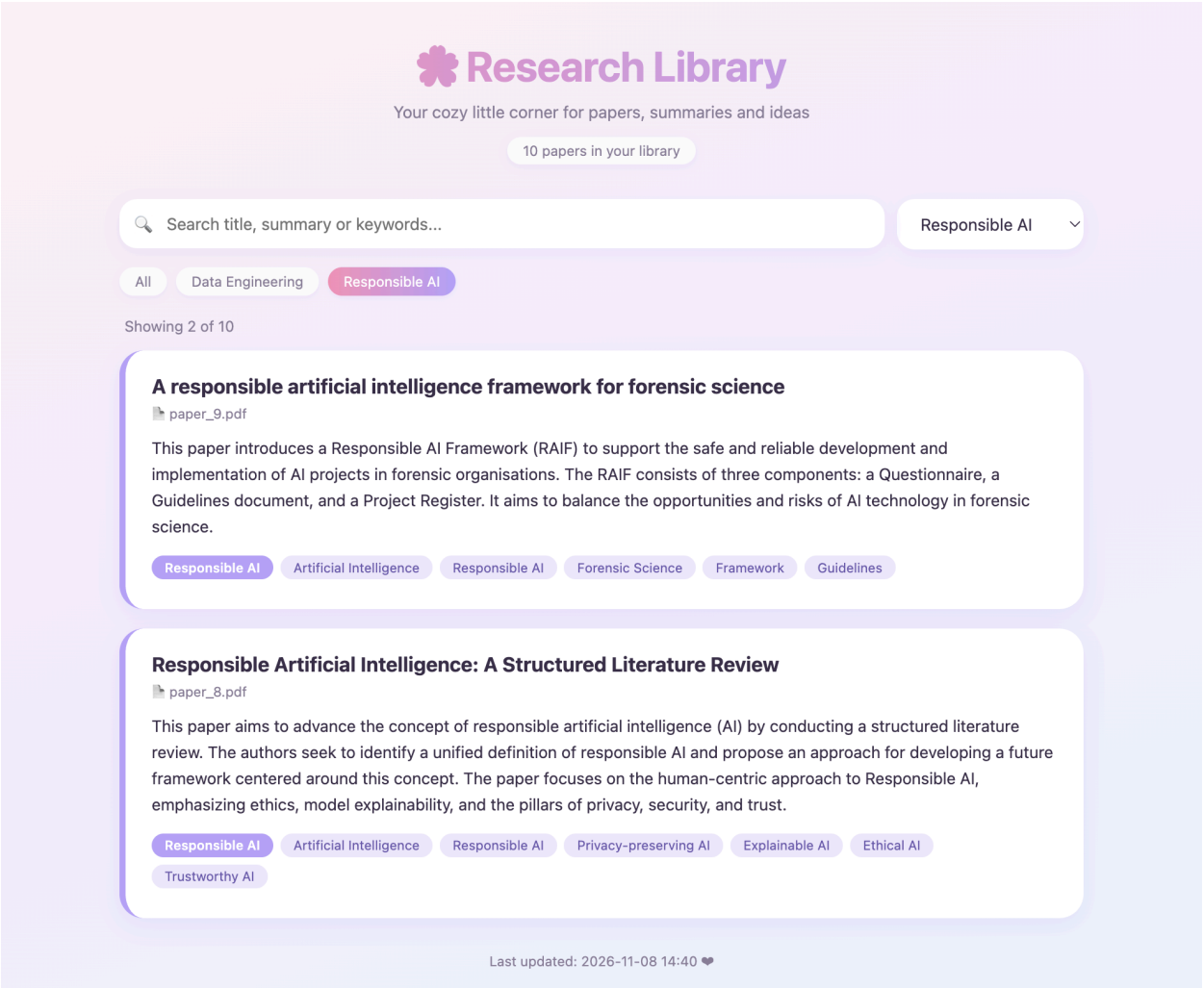
```
aysha@Umar-Hashmi:~$ sqlite3 -header -column ~/airflow_data/papers.db "SELECT filename, title, category FROM pape
rs;"
filename                    title                                                    category
--------------------------  -------------------------------------------------------  ---------------
Assignment 3.pdf            INFO-H420 Process Modeling, Data Pipelines, and Responsible AI  Data Engineering
INFO-H420-Assignment2.pdf   Management of Data Science and Business                   Data Engineering
System_R_Explained_Simply.pdf  System R, Explained Simply                            Data Engineering
```

## 3.4 HTML Generation and Discord Notifications

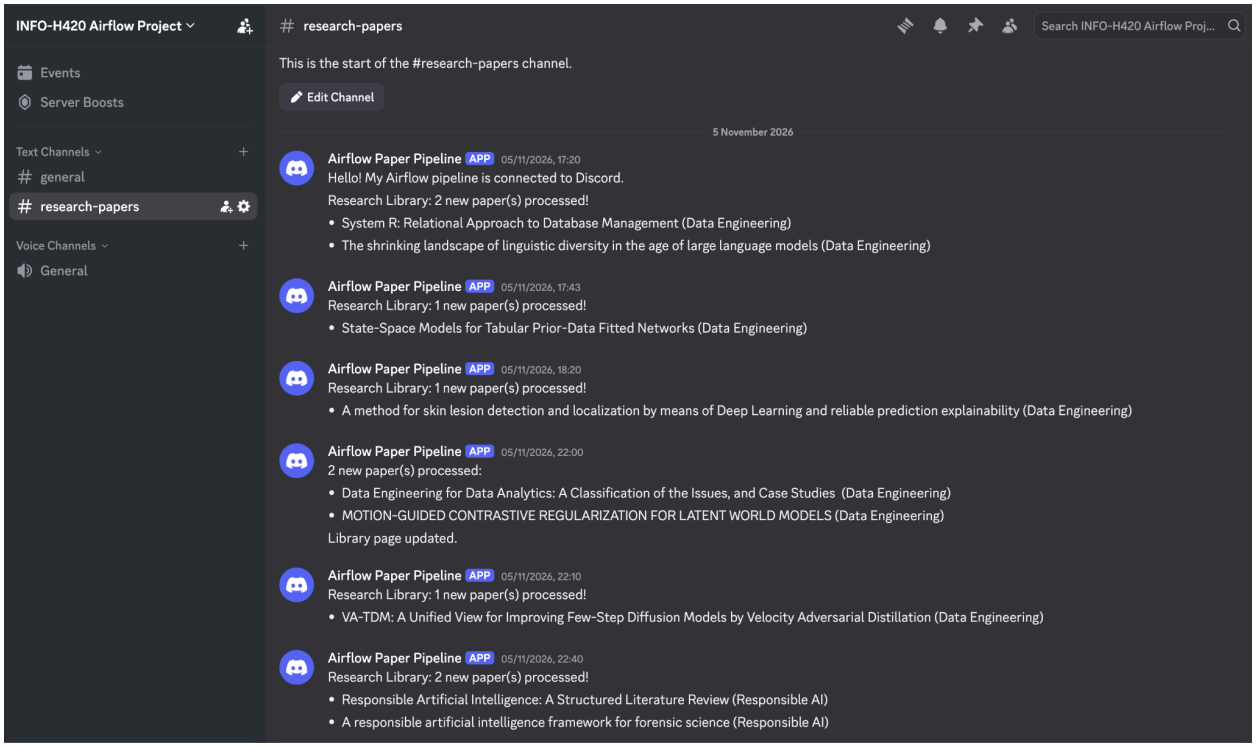
After processing the new papers, the build_html task retrieves all stored metadata from the SQLite database and regenerates a single static index.html page in the Google Drive-synchronized output/ folder.




The page displays each paper's title, summary, keywords, and category. It also provides a search box for filtering papers by title, summary, or keywords, along with a category dropdown for easier navigation. These features are implemented using JavaScript without external libraries.



Once the HTML page has been generated successfully, the final Airflow task sends a Discord notification through a webhook.(The webhook URL is read from the DISCORD_WEBHOOK_URL environment variable and is never committed to Git.) The notification lists the newly processed papers and their corresponding categories. As It's shown in the following screenshot.




# 4. Experimental Validation and Results

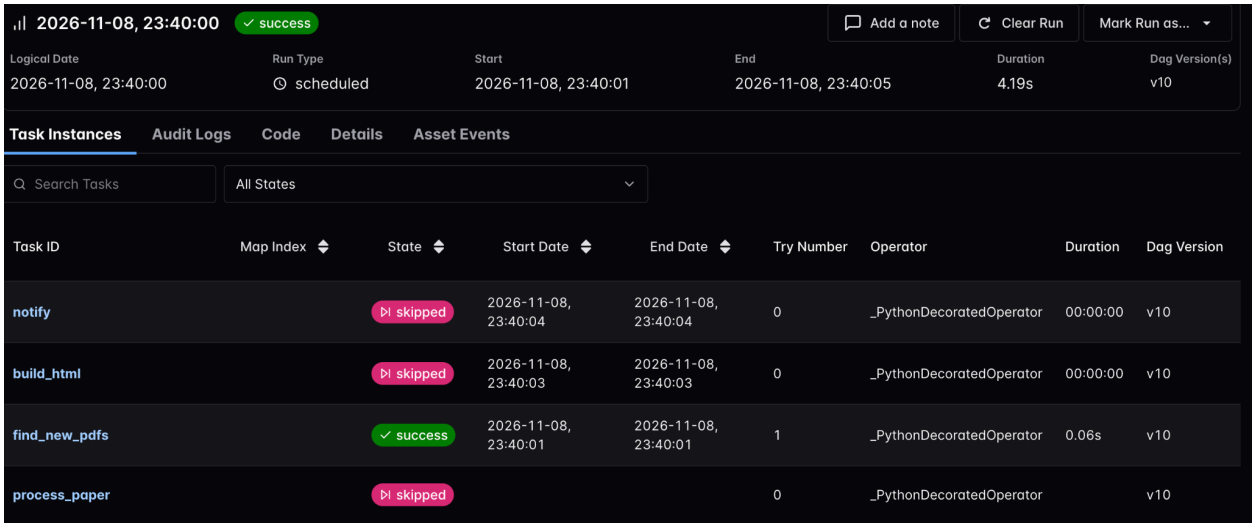
To verify the implementation, the pipeline was tested under different conditions. The experiments focus on correct execution, processing of new files, duplicate detection, and generation of the expected outputs.

## 4.1 Initial Processing of Multiple PDFs

The first experiment evaluates the pipeline's ability to process multiple research papers in a single DAG execution. Which you can see the logs and the discord messages in the previous figure in section 3.4 the first two papers are System R and The shrinking landscape of linguistic.

## 4.2 Duplicate Detection and Idempotence

To verify duplicate detection, the DAG was executed again without adding new PDFs. The find_new_pdfs log reported "Found 10 PDFs, 0 new: []", confirming that all detected files had already been processed. As shown in the following figure, find_new_pdfs completed successfully, while process_paper, build_html, and notify were skipped. This demonstrates that the pipeline avoids duplicate processing and unnecessary task execution.

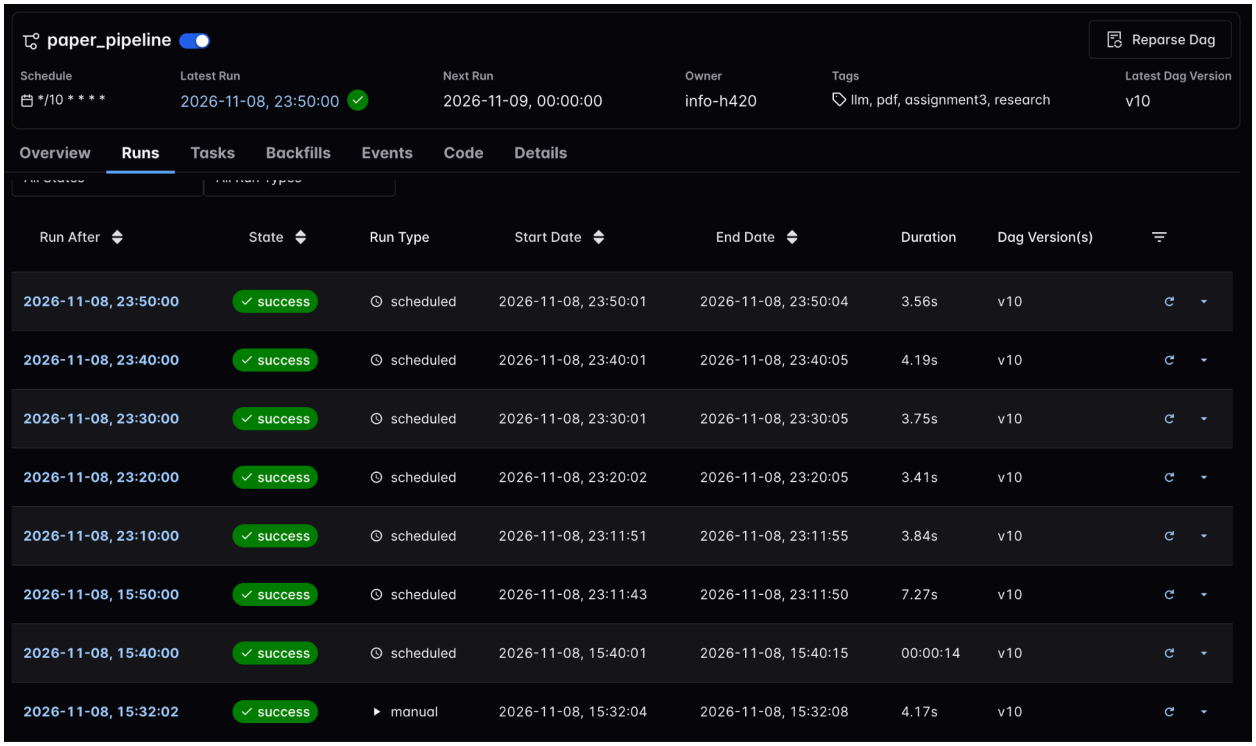



## 4.3 Incremental Processing of New PDFs

To evaluate incremental processing, new PDFs were added to the input folder after previous executions. As shown in Discord messages Figure in section 3.4 , the Discord notifications confirm that the pipeline successfully processed additional papers in separate runs, including individual new documents. This demonstrates that the workflow can detect and process newly added PDFs without requiring the entire collection to be processed again.

## 4.4 Automatic Scheduling and End-to-End Verification

The DAG is configured to execute automatically every 10 minutes, allowing the pipeline to check for newly added PDFs without manual intervention, provided Airflow and the required local services are running. As shown in the following Figure, the Airflow execution history confirms that multiple runs were automatically triggered at 10-minute intervals, with all displayed runs completing successfully. The scheduled run type distinguishes these executions from manually triggered runs. These results demonstrate that the scheduling mechanism operates correctly after manually triggered.




## 5. Conclusion

In this project, we developed an automated research-paper processing pipeline using Apache Airflow and a locally hosted LLM. The system successfully integrates PDF conversion, metadata extraction, SQLite storage, HTML generation, and Discord notifications. The experimental results demonstrate that the workflow can process multiple documents, avoid duplicate processing, and execute automatically according to its schedule. Overall, the project demonstrates how Airflow can be used to coordinate different components and automate the organization of research papers into a searchable library.